

Language Models and Agents

Individual Project Report

2023102073
Harshita Kumari

September 2026

Abstract

Two independent approximately 25M-parameter decoder-only Transformer language models were built and trained entirely from scratch for Hindi and Nepali. The models use separate corpora, tokenizers, and weights. Approximately 500M tokens were used for pretraining in each language. The models were evaluated using perplexity, bits-per-byte, generation metrics, and attention analysis, followed by finetuning on a synthetic reasoning task. Hindi obtained a test perplexity of 31.96 and 36.75% reasoning accuracy after finetuning, while Nepali obtained a test perplexity of 40.33 and 8.75% reasoning accuracy.

1 Phase 1: Data Collection and Tokenizer

1.1 Data Collection

The corpus was built using a combination of FineWeb2, IndicCorp, Sangraha, Wikipedia, and several manually collected web sources. For the manual collection, we used sitemaps and archives to find relevant pages and applied rate limits at the domain level. Sources that explicitly restricted AI-related crawling were not included.

Table 1: Final corpus statistics

Metric	Hindi	Nepali
Total tokens	499,999,997	499,302,601
Manual token fraction	20.29%	19.89%
Manual tokens	101,450,874	99,302,601
Downloaded tokens	398,549,123	400,000,000

The Nepali corpus fell slightly short of the 500M-token target, by 697,399 tokens (0.14%).

1.2 Data Processing Pipeline

Before training the tokenizers, both corpora were processed using the same four-stage pipeline, with the entire process carried out independently for each language.

1. Cleaning, deduplication, and sampling. The data was processed in this order:

- Unicode normalization and mojibake correction** using `ftfy`. This helped fix encoding issues that can occur in scraped or downloaded text. NFC normalization ensured that visually identical characters were represented in the same way before duplicate removal.
- Script filtering.** We kept lines containing at least 60% Devanagari characters to remove irrelevant non-Devanagari boilerplate while still keeping naturally occurring mixed-script sentences.
- Whitespace and punctuation normalization.** Removed extra whitespace and inconsistent punctuation to make text more uniform.
- Exact duplicate removal.** We used SHA-256 hashes of the normalized, lowercased, and whitespace-collapsed text to identify duplicates. Manual documents were given priority when a manual and downloaded document had the same hash.
- Minimum-length filtering.** Documents containing fewer than 20 characters after cleaning were removed.
- Sampling to the target token count.** After cleaning, the remaining documents were sampled until the required token target was reached, while following the specified manual-to-downloaded data ratio.

2. Splitting. The data was split at the **document level** rather than splitting individual sentences or portions of documents. This prevents parts of the same document from appearing in both the training and evaluation sets and helps avoid data leakage.

Table 2: Train/validation/test split sizes (document counts)

Language	Train		Val	Test
Hindi	1,340,887 (98.00%)	13,682 (1.00%)	13,684 (1.00%)	
Nepali	1,855,143 (98.00%)	18,930 (1.00%)	18,931 (1.00%)	

The pipeline is summarized as:

raw $\rightarrow$ *clean/dedup/filter/sample* $\rightarrow$ *split(train/val/test)* $\rightarrow$ *traintokenizer(trainsplitonly)*.

1.3 Tokenizer Training

Algorithm: SentencePiece BPE. We used Byte-Pair Encoding (BPE) to build the tokenizer. It starts with individual characters as the basic vocabulary and gradually combines the most frequently occurring adjacent pieces into larger pieces. This process continues until the vocabulary reaches the target size of 8,000 tokens. As a result, frequently occurring words or parts of words can be represented using a single token, while less common words are broken down into smaller pieces. This also helps keep the unknown-token rate low, since unseen words can still be represented using smaller subword pieces or individual characters.

We used SentencePiece because it does not require the text to be split into words before tokenization. This is useful for Devanagari text, where whitespace does not always correspond perfectly to word boundaries.

Table 3: Final tokenizer statistics

Metric	Hindi	Nepali
Vocabulary size	8,000	8,000
Fertility (tokens/word)	1.380	1.536
Unknown-token rate	0.261%	0.190%

Choosing the vocabulary size. To decide the vocabulary size, we tested different values ranging from 2,000 to 32,000 on a validation sample. The unknown token rate remained below 0.33% for both languages across all vocabulary sizes, so this was not a major factor in our decision.

The main trade off was between fertility and the number of parameters required by the embedding table. Since the embedding size grows linearly with the vocabulary size ($\text{vocab_size} \times d_{\text{model}}$), using a much larger vocabulary increases the parameter cost significantly. For example, increasing the vocabulary from 8K to 32K improves Hindi’s fertility by only about 13% (1.38 to 1.53 tokens/word), while the embedding table increases from 16.38% to 65.54% of the 25M-parameter budget. Based on this trade-off, we selected a vocabulary size of 8,000, where the improvement from using a larger vocabulary was no longer worth the additional parameter cost.

Table 4: Vocab-size sweep

Vocab size	Hindi		Nepali		Emb. % of 25M
	Fertility	UNK%	Fertility	UNK%	
2,000	1.7747	0.218%	2.1043	0.139%	4.10%
4,000	1.5455	0.250%	1.7778	0.164%	8.19%
8,000	1.3795	0.280%	1.5364	0.190%	16.38%
16,000	1.2667	0.305%	1.3728	0.213%	32.77%
24,000	1.2219	0.316%	1.3022	0.224%	49.15%
32,000	1.1988	0.322%	1.2621	0.231%	65.54%

Tokenizer vocab-size sweep: fertility (tokens/word) / UNK-rate / cost trade-off
SentencePiece BPE, both languages, systematic 150K-doc sample of the train split; chosen vocab size = 8,000 marked

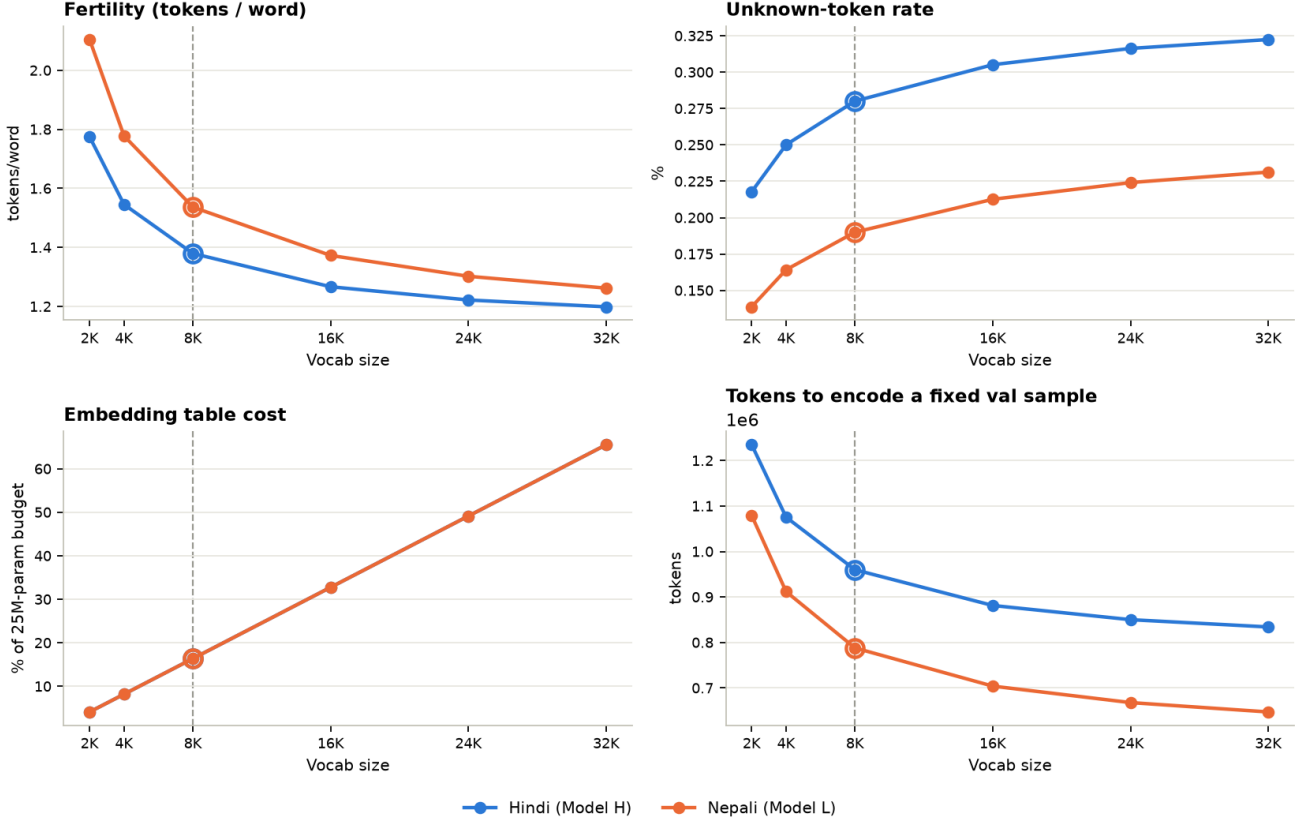


Figure 1: Fertility (tokens/word), UNK rate, and embedding cost vs. vocab size (8,000 marked).

2 Phase 2: Model, Pretraining and Evaluation

2.1 Architecture

Both models use the same architecture while keeping their weights fully independent.

Table 5: Model configuration

Parameter	Value
Vocabulary	8,000
Model dimension d_{model}	512
Transformer layers	7
Attention heads	8
Head dimension d_k	64
FFN dimension	2,048
Context length	512
Dropout	0.1
Output head	Tied to embeddings
Total parameters	26,425,856

Positional embeddings. Learned absolute positional embeddings are used instead of sinusoidal or RoPE: at this scale, with a fixed 512-token context, a learned 512×512 table costs only $\sim 1\%$ of the model and is simpler to verify, at the cost of no length extrapolation beyond 512 tokens.

Causal self-attention. Implemented from four independent W_Q, W_K, W_V, W_O projections (not a fused QKV matrix, for readability):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V,$$

with $h = 8$ heads of $d_k = 512/8 = 64$ each. The $1/\sqrt{d_k}$ scale keeps pre-softmax logit variance constant regardless of head size – without it, variance grows linearly with d_k and pushes softmax toward a near-one-hot regime with vanishing gradients. M is an additive causal mask ($0 / -\infty$), and its correctness was verified empirically (perturbing only the last token of a random sequence and confirming every earlier logit is unchanged): max logit difference was exactly 0.0 across 5 trials.

Transformer block. Pre-norm self-attention (residual) $\rightarrow$ pre-norm feed-forward (residual). Pre-norm was chosen over post-norm for training stability – post-norm GPT-style models are markedly more sensitive to learning-rate/warmup, a risk not worth taking on a limited, hard-to-diagnose Colab compute budget. The FFN uses GELU with inner dimension $4 \times d_{\text{model}} = 2048$.

Weight tying. The output head is tied to the token embedding matrix, saving an entire $8000 \times 512 = 4,096,000$ -parameter matrix – 13.4% of the model (30.5M untied vs. 26.4M tied), a meaningful fraction at this parameter budget.

Table 6: Exact parameter breakdown (sums to the reported total)

Component	Parameters
Token embedding (tied with output head)	4,096,000
Positional embedding	262,144
$7 \times$ Transformer block (3,152,384 each)	22,066,688
Final LayerNorm	1,024
Total	26,425,856

Depth vs. width. With an 8,000-token vocabulary, the embedding tables cost a near-fixed $\sim 4.4\text{M}$, leaving $\sim 22\text{M}$ for the Transformer stack. A narrower-but-deeper configuration (512 dim, 7 layers) was chosen over wider-shallower because depth composes hierarchical/longer-range structure more effectively per parameter at this scale, while keeping per-step compute tractable for a Colab session. $d_{\text{model}} = 512$ and $n_{\text{heads}} = 8$ were kept as well-precedented defaults rather than swept, since architecture search was out of scope.

2.2 Pretraining Procedure

Rather than going through the corpus sequentially from beginning to end, we used random sampling during training. At each step, 32 random 512-token windows were selected from the tokenized corpus, with replacement. This means that the same part of the corpus could be sampled more than once, while some parts might not be seen during the training run.

Gradients were accumulated over 4 micro-batches before updating the model. Therefore, each optimizer step processed

$$32 \times 4 \times 512 = 65,536 \text{ tokens.}$$

Training was carried out for a fixed total of 8,000 steps.

Why random sampling with replacement instead of epochs. We used a fixed token budget of approximately 500M tokens rather than defining training in terms of epochs. In fact, the Hindi corpus alone contains around 675M tokenized tokens, which is already larger than the 524M-token training budget. Therefore, training for a complete epoch would not be possible within the given budget.

This left us with two possible approaches: making one deterministic pass through a shuffled version of the corpus and stopping when the token budget was reached, or randomly sampling windows with replacement. We chose the second approach because it does not require creating and storing a precomputed shuffle or index for the entire corpus. It also makes it easy to resume training after an interruption using just the training step count. This was particularly useful for the Nepali model, whose training had to be restarted four times. Random sampling with replacement is also consistent with the approach commonly used in large-corpus language model training, such as GPT-2, GPT-3, and nanoGPT, when the training budget is smaller than the available corpus.

Table 7: Pretraining hyperparameters

Setting	Value
Optimizer	AdamW, $\beta = (0.9, 0.95)$
Peak learning rate	3×10^{-4}
Schedule	linear warmup (200 steps) $\rightarrow$ cosine decay to 10% of peak
Weight decay	0.1, decoupled, ≥ 2 D matrices only
Gradient clipping	max norm 1.0
Batch $\times$ accumulation $\times$ block	$32 \times 4 \times 512 = 65,536$ tokens/step
Total steps	8,000 (≈ 524 M tokens)
Checkpointing	every 200 steps, resumable (<code>latest.pt</code> + <code>best.pt</code>)

Table 8: Pretraining results

Metric	Hindi	Nepali
Tokens seen	524,288,000	524,288,000
Final train loss	3.5647	3.7242
Final validation loss	3.4717	3.6646
Validation perplexity	32.19	39.04

2.3 Intrinsic Evaluation

Perplexity was calculated as

$$PPL = e^{\bar{\ell}},$$

where $\bar{\ell}$ is mean cross-entropy in nats per token. Bits-per-byte was calculated as

$$BPB = \frac{\sum_i NLL_i}{\ln(2) \times \text{UTF-8 bytes}}.$$

Table 9: Test-set intrinsic evaluation

Metric	Hindi	Nepali
Test tokens	6,630,400	7,754,752
Cross-entropy	3.4646	3.6971
Perplexity	31.96	40.33
Bits-per-byte	0.5259	0.4810

Since the Hindi and Nepali tokenizers split text into different numbers of tokens, their token-level perplexities are not directly comparable. BPB provides an additional measure that evaluates performance at the byte level.

2.4 Generation Evaluation

Generation was evaluated using held-out prefixes with greedy decoding and temperatures 0.5, 1.0, and 1.5. BLEU-4, chrF++, ROUGE-L, Distinct-1, and repetition were measured.

Table 10: Generation metrics

Model	Setting	BLEU	chrF++	ROUGE-L	D-1	Rep.
H	Greedy	1.76	13.88	.255	.132	.603
H	$T = 0.5$	1.49	14.91	.247	.223	.222
H	$T = 1.0$	.86	16.52	.252	.463	.003
H	$T = 1.5$	.12	14.26	.233	.676	.000
L	Greedy	1.75	14.60	.212	.176	.557
L	$T = 0.5$	1.91	16.90	.242	.336	.169
L	$T = 1.0$	.84	18.30	.233	.598	.002
L	$T = 1.5$	.10	15.97	.228	.754	.000

Greedy decoding showed substantial repetition. Higher temperature reduced repetition and increased diversity, but very high temperature also reduced reference overlap.

2.5 Attention Analysis

Attention entropy and mean attention distance were measured for every (layer, head) pair, on 200 held-out test sentences (up to 128 tokens each):

$$H_{l,h} = -\frac{1}{T} \sum_t \sum_k p_{t,k}^{(l,h)} \log p_{t,k}^{(l,h)}, \quad D_{l,h} = \frac{1}{T} \sum_t \sum_k p_{t,k}^{(l,h)} |t - k|,$$

where $p^{(l,h)}$ is the softmax attention weight matrix for layer l , head h . Lower entropy means more peaked (focused) attention; larger distance means the head attends further from the query position.

Table 11: Mean attention entropy and distance by layer (averaged over 8 heads)

Layer	Entropy (nats)		Mean distance (tokens)	
	Hindi	Nepali	Hindi	Nepali
0 (early)	3.29	3.31	20.39	20.51
1	3.36	3.36	23.40	24.13
2	2.92	3.11	14.81	17.94
3	2.47	2.30	8.67	6.46
4 (most local)	1.74	1.60	6.22	5.11
5	2.08	2.19	13.00	15.34
6 (late)	2.20	2.20	31.29	31.58

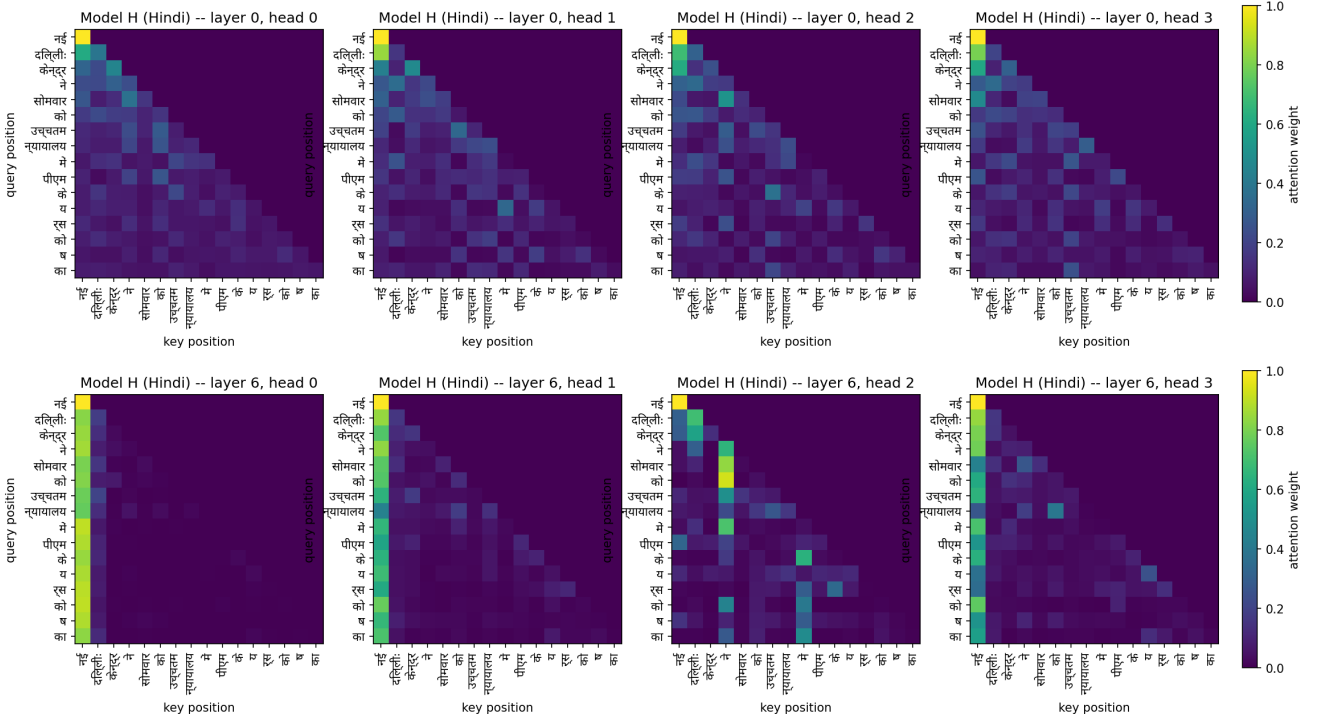


Figure 2: Model H (Hindi): layer 0 (early, diffuse) vs. layer 6 (late, sink-dominated) attention heatmaps, heads 0–3.

Early layers (0–1): broad and less specialized attention. In the first few layers, the attention entropy remains relatively high, around 3.3–3.4 nats, while the mean attention distance is also fairly large (~20–24 tokens). This suggests that the heads are not yet strongly specialized and tend to spread their attention across a wider part of the context. At this stage, the model has not built up enough higher-level representations to focus strongly on particular positions.

Middle layers (3–4): more local and positional attention. In the middle layers, the attention patterns become much more focused. Entropy drops to as low as 0.19–0.75 nats for some heads, and the mean attention distance decreases to around 5–9 tokens. This indicates that some heads start focusing mainly on nearby tokens and positional relationships. For Hindi and Nepali, these local dependencies can include adjacent-word relationships and attached postpositions or case markers. Such short-range patterns are useful for predicting the next token, so the model appears to dedicate some of its attention capacity to these local relationships.

Layer 6 (late): large distances caused mainly by an attention sink. In the final layer, the mean attention distance increases again to around 31 tokens, which is the highest among the layers. However, this does not necessarily mean that the model is performing more long-range reasoning. The heatmaps suggest that the increase is mainly caused by an **attention sink**. Several heads put a large amount of their attention on a fixed early token, usually position 0, from many different query positions.

This can happen because the attention weights have to sum to 1 across all positions. When a head does not have a particularly relevant token to focus on, it may place much of its attention on a fixed token that is always present. As a result, the average distance becomes larger even though the head is not necessarily using meaningful long-range information. This is why the heatmaps are important for interpreting the results rather than relying only on the entropy and distance values. At the same time, a few heads in this layer do show genuine long-range, content-based attention rather than the attention-sink pattern.

3 Phase 3: Reasoning Finetuning

3.1 Synthetic Dataset

Each language’s reasoning finetune set was generated programmatically by a shared, language-agnostic generator (`reasoning_gen.py`), with per-language word lists and templates supplying all actual text – so the two languages share generation *logic* but no content. Three task types are covered:

- **Direct:** compare 2 entities on one attribute (age, height, price, or quantity) – answer is the greater, smaller, or “both equal”.
- **Transitive:** 3 entities chained $A \sim B \sim C$ – answer is the overall extreme (min/max).
- **Multihop:** 3 entities chained, but the question asks about the *derived*, non-adjacent relation between A and C – the only task type that requires actually chaining both stated facts rather than reading off an implied order.

Table 12: Reasoning dataset statistics (identical for Hindi and Nepali)

	Train	Val	Test
Total examples	6,000	800	800
Direct	2,024	277	368
Transitive	1,991	258	227
Multihop	1,985	265	205
Age	1,515	216	260
Height	1,519	189	263
Price	1,471	199	156
Quantity	1,495	196	121

Leakage control. Both entity names *and* sentence templates were split into disjoint train/test pools before generation, so every test example uses an entity name *and* a template phrasing the model never saw during finetuning – 6 held-out person names and 3 held-out object names per language, with at least one template per task type reserved for test only.

Table 13: Example reasoning prompts (native script)

Lang.	Task	Prompt	Answer
Hindi	Direct	अगर राम की उम्र 55 वर्ष हो और सीता की उम्र 66 वर्ष हो, तो किसकी उम्र ज़्यादा होगी? उत्तर:	सीता
Hindi	Multihop	राम की उम्र, सीता की उम्र से कम है। सीता की उम्र, सुरेश की उम्र से कम है। राम और सुरेश में से किसकी उम्र ज़्यादा है? उत्तर:	सुरेश
Nepali	Direct	यदि विकासको उमेर 23 वर्ष छ र सीताको उमेर 23 वर्ष छ भने, कसको उमेर बढी होला? जवाफ:	दुवै बराबर छन्
Nepali	Multihop	चशमाको मूल्य, कलमको मूल्य भन्दा कम छ। कलमको मूल्य, पङ्खाको मूल्य भन्दा कम छ। चशमा र पङ्खा मध्ये कसको मूल्य बढी छ? जवाफ:	पङ्खा

3.2 Finetuning

Finetuning started from each language’s own pretrained checkpoint (weights only – optimizer/scheduler state was reinitialized fresh, and the step counter restarts at 0). Each example is tokenized as prompt + answer + EOS, and

the loss is masked so only the answer span (not the prompt) is supervised. Unlike pretraining, each step draws one batch of 32 examples directly (no gradient accumulation), sampled with replacement from the 6,000-example train set – the same resumable-by-step-counter philosophy as pretraining.

Table 14: Finetuning hyperparameters

Setting	Value
Optimizer	AdamW, $\beta = (0.9, 0.95)$
Peak learning rate	2×10^{-5}
Schedule	linear warmup (50 steps) $\rightarrow$ cosine decay to 10% of peak
Batch size	32 (no gradient accumulation)
Max sequence length	96 tokens
Gradient clipping	max norm 1.0
Loss	masked cross-entropy, answer span only
Steps run	3,000 (Hindi) / 5,000 (Nepali)

Reasoning finetuning: train vs. validation loss (masked cross-entropy, answer span only)

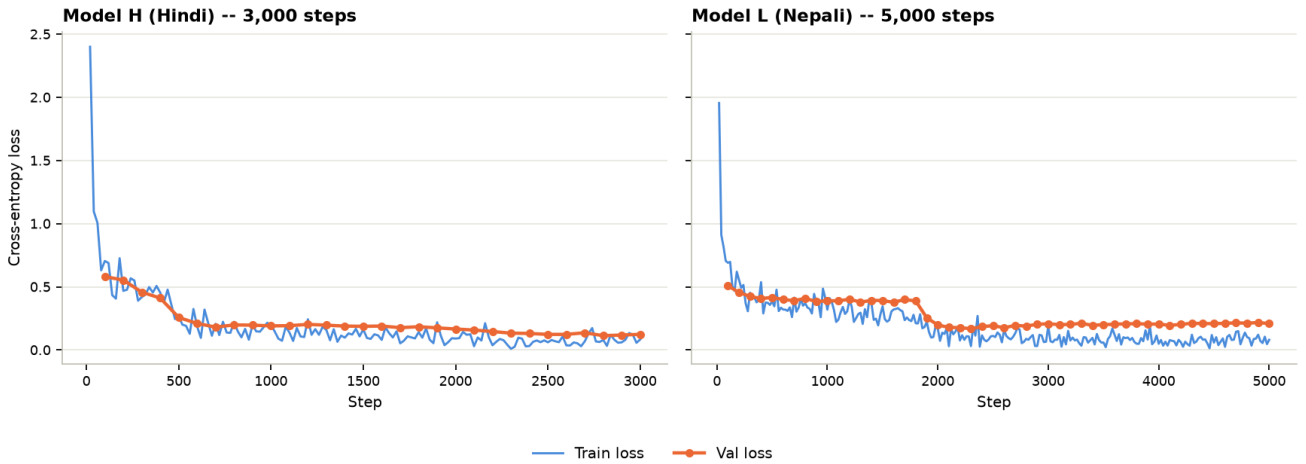


Figure 3: Reasoning finetuning train/validation loss (masked cross-entropy, answer span only). Hindi converges smoothly; Nepali shows a plateau followed by a step-change drop around step $\sim 1,900$, plausibly from the cosine-decay schedule being recomputed against a new `max_steps` each time the run was resumed with an extended step budget.

Nepali was extended from 1,000 (config default) to 3,000, then to 5,000 steps specifically to test whether its accuracy gap versus Hindi was a step-budget artifact or a real ceiling

3.3 Reasoning Results

Table 15: Answer-restricted accuracy: pretrained vs. finetuned (same held-out test set)

Task	Hindi		Nepali	
	Pretrained	Finetuned	Pretrained	Finetuned
Overall	32.1%	22.0%	36.6%	11.9%
Direct	36.7%	11.7%	40.8%	11.7%
Transitive	25.1%	26.4%	30.8%	0.9%
Multihop	31.7%	35.6%	35.6%	24.4%

The pretrained models achieved 0% exact-match accuracy. Finetuning substantially improved performance, with a larger improvement for Hindi.

Increasing Nepali finetuning from 3,000 to 5,000 steps did not close the gap, suggesting that insufficient optimization steps alone do not explain the difference.

3.4 Failure Analysis

Hindi errors generally remained within the entities appearing in the prompt. Nepali sometimes generated entity names absent from the current prompt but present in the training pool, indicating a possible entity-binding or grounding problem.

Attention statistics also changed after finetuning, with overall entropy decreasing in both models. These changes show that finetuning altered attention organization, although they do not by themselves establish the cause of the reasoning gap.

4 Examples:

Table 16: Example prompts: pretrained vs. finetuned predictions (real model outputs)

Lang.	Task	Prompt	Answer	Pretrained pred.	Finetuned pred.
Model H (Hindi)	Multihop	राम की उम्र, सीता की उम्र से कम है। सीता की उम्र, सुरेश की उम्र से कम है। राम और सुरेश में से किसकी उम्र ज्यादा है? उत्तर:	सुरेश	राम की उम्र, राम की उम्र,	सुरेश
Model H (Hindi)	Transitive	पता है कि राम की ऊँचाई, विकास की ऊँचाई से कम है, और विकास की ऊँचाई, सीता की ऊँचाई से कम है। इनमें सबसे कम ऊँचाई किसकी है, बताइए। उत्तर:	राम	- "अब तक, राम की	राम
Model H (Hindi)	Direct	अगर विकास की ऊँचाई 160 सेंटीमीटर हो और सुनीता की ऊँचाई 108 सेंटीमीटर हो, तो किसकी ऊँचाई कम होगी? उत्तर:	सुनीता	-पश्चिमी भाग में, यह एक	दोनों बराबर हैं
Model L (Nepali)	Multihop	चश्माको मूल्य, कलमको मूल्य भन्दा कम छ। कलमको मूल्य, पङ्खाको मूल्य भन्दा कम छ। चश्मा र पङ्खा मध्ये कसको मूल्य बढी छ? जवाफ:	पङ्खा	"अनलाइन" को मूल्य,	पङ्खा
Model L (Nepali)	Direct	यदि विकासको उमेर 23 वर्ष छ र सीताको उमेर 23 वर्ष छ भने, कसको उमेर बढी होला? जवाफ:	दुवै बराबर छन्	(empty)	दुवै बराबर छन्
Model L (Nepali)	Direct	यदि मनोजको उचाइ 160 सेन्टिमिटर छ र सुनिताको उचाइ 108 सेन्टिमिटर छ भने, कसको उचाइ कम होला? जवाफ:	सुनिता	(empty)	सुनिल

5 Conclusion

Two independent decoder-only Transformer language models were implemented and trained from scratch for Hindi and Nepali. Both used approximately 500M-token corpora, 8,000-token vocabularies, and the same approximately 25M-parameter architecture.

The experiments show that intrinsic language modeling, generation, attention organization, and reasoning transfer measure different properties. Hindi obtained 36.75% reasoning accuracy after finetuning, compared with 8.75% for Nepali, while intrinsic metrics showed a more nuanced picture because of tokenizer differences.